Algorithms – mid-term fact sheet.

Definitions:

- *Undirected graph*: G=(V,E), V=vertices, E=undirected edges.
- *Directed graph*: G=(V,E), V=vertices, E=directed edges.
- Notation: |V|=n, |E|=m.
- A *simple* graph: a graph with no parallel edges or self-loops.
- *Bipartite* graph: G=(V_1,V_2,E). For each edge e=(u,v), u$\in V_1$, v$\in V_2$
- *Complete* graph (*clique*) of size n. A graph in which |V|=n and E=V×V (but no self loops).
- *Complete bipartite*: A bipartite in which E= $V_1 \times V_2$.
- A *path* is a sequence of vertices $v_0,v_1, \ldots,v_k$ such that $(v_i,v_{i+1}) \in E$ for all $0 \leq i < k$
- An undirected graph is *connected* if for any pair of vertices u,v, there is a path between u and v.
- A directed graph is *strongly connected* if for any pair of vertices u,v, there is a directed path from u to v.
- *Undirected Tree*: An undirected graph is a *tree* if it is connected and acyclic
- *Directed Tree:* A directed graph is a *directed tree* if it has a *root* and its underlying undirected graph is a tree.
- r$\in$V is a *root* if every vertex v$\in$V there is a directed path from r to v.
- *Topological sort* of a directed graph: a linear ordering of the vertices such that for any edge $(v,w) \in E$, *v* precedes *w* in the ordering.
- An Euler path is a path that contains each edge exactly once.
- An Euler cycle is an Euler path that starts and ends at the same vertex.
- Let $\Sigma = \{0, 1, ..., \sigma - 1\}$ be an alphabet. Let $L = \sigma^n$. A *de Bruijn sequence* is a (circular) sequence $a_0a_1 \ldots a_{L-1}$ over Σ such that for every word w of length *n* over Σ there exists a unique *i* such that $a_i a_{i+1} \ldots a_{i+n-1} = w$, the computation of the indices is modulo *L*.

Facts:

- Let G=(V,E) be an undirected graph, then $2|E| = \Sigma_v \deg(v)$.
- Any directed acyclic graph has a topological sort.
- An undirected graph has an Euler cycle if and only if it is connected and all its vertices have even degrees
- A directed graph has an Euler path which is not a cycle if and only if it is connected and exactly two of its vertices have odd degrees
- A Hamiltonian path (cycle) goes through each vertex exactly once (and returns to the starting vertex).
- The digraph $G_{\sigma,n}(V, E)$ built for constructing a De-Bruin sequence for σ and *n* :
  1. *V* is the set of all σ^{n-1} words of length $n - 1$ over Σ.
  2. *E* is the set of all σ^n words of length *n* over Σ. The edge $b_1b_2 \ldots b_n$, starts at vertex $b_1b_2 \ldots b_{n-1}$ and ends at vertex $b_2b_3 \ldots b_n$.

<u>Algorithm for topological sort.</u> Time complexity O(n+m):

- Store each vertex's In-Degree in an array D
- Initialize queue with all "in-degree=0" vertices
- While there are vertices remaining in the queue:
  - Dequeue and output a vertex
  - Reduce In-Degree of all vertices adjacent to it by 1
  - Enqueue any of these vertices whose In-Degree became zero.
- If all vertices are output then success, otherwise there is a cycle.

DFS(i: vertex)
```
mark i;
for each j adjacent to i do
  if j is unmarked then DFS(j)
```

BFS (centralized version):

1. Label vertex s with 0.
2. $i \leftarrow 0$
3. Find all unlabeled vertices adjacent to at least one vertex labeled i. If none are found, stop.
4. Label all the vertices found in (3) with i + 1.
5. $i \leftarrow i + 1$ and go to (3).

BFS (with queue):
```
Initialize E' to be empty;
Initialize Q to be empty;
Enqueue(1,Q) and mark 1;
while (Q is not empty) do
   i := Dequeue(Q);
   for each j adjacent to i do
      if j is not marked then
         add {i,j} to E';
         mark j;
         Enqueue(j,Q);
```

General shortest path method:

Init:
```
s.d = 0
s.π = null
for all other v
    v.d = infinity
    v. π = null
```

relax(u,v):
```
if v.d > u.d + w(u,v)
     v.d = u.d+w(u,v)
     v. π = u
```

repeat
```
choose some edge u,v
relax(u,v)
```

Shortest Path Algorithm for Acyclic Graphs: Time complexity O(n+m):

1.Topologically sort the vertices.
2. According to the sorted order:
```
for all u
      for all v in u.nbrs relax(u,v)
```

Dijkstra Algorithm for Graphs with non-negative weights:

Q is a priority queue where priority is according to the value of d.

```
Q = V(G)
while Q is not empty
   u = Q.dequeue (u is a vertex with minimal d value)
   for all v in u.nbrs
      relax(u,v)
```

Time complexity: $O((n + m) \log n))$ or $O(n^2)$ or $O(m + n \log n)$ using Fibonacci heaps.

Bellman Ford Algorithm for general graphs: Time complexity: O(nm)

```
Repeat n-1 times
   for all edges (u,v)  relax(u,v)
```

Floyd -Warshall Algorithm – all-pairs, general graph. Time complexity $O(n^3)$

1. Init $\delta^0(i, j)$
2. $k \leftarrow 1$
3. For every $1 \le i, j \le n$ compute
   $\delta^k(i, j) \leftarrow \text{Min} \{ \delta^{k-1}(i, j), \delta^{k-1}(i, k) + \delta^{k-1}(k, j)\}$.
4. If $k = n$, stop. If not, increment k and go to step 3.

$$\delta^0(i,j) = \begin{cases} w(e) & \text{if } (i,j) \in E, \\ \infty & \text{if } (i,j) \notin E. \\ 0 & \text{if } i = j \end{cases}$$